STM32 Firmware

GENERATING 32-BIT STM32 UNIQUE ID

The **STM32 Unique ID** is 96 bits long (12 bytes). For most applications, it is not useful to have such a long unique ID because of a limited number of units in circulation. However, it might be extremely important for you to have a unique ID that does not match with another device in the lot – at least has a VERY LOW chance of matching another device.

The expectations when trying to figure out this hashing method were the following:

- Must be **fast and easy**
- Should take **less than 512 bytes flash and 128 bytes RAM**
- Should be **hardware-independent**
- **Single C and H file** – no mess
- 32-bit unique ID must not repeat in a lot of, say, 10,000 consecutive UID pcs.

Structure of the 96-bit unique ID

The 96-bit unique ID present on the STM32 chips (at least the STM32L4 series) is structured in a specific way. This information is not specified for all STM32 chips, but I would assume that this is how their UIDs are put together as well.

The STM32L4 reference manual **(RM0351)** breaks down the 96-bit unique ID structure as follows:

- **UID[95:64]:** LOT_NUM[55:24] – Lot number (ASCII encoded)

- **UID[39:32]:** WAF_NUM[7:0] – Wafer number (8-bit unsigned number)
- **UID[63:40]:** LOT_NUM[23:0] – Lot number (ASCII encoded)
- **UID[31:0]:** X and Y coordinates on the wafer

As you can see, none of the above data fields can be extracted and put together as a 32-bit STM32 unique ID without the risk of repitition of the 32-bit UID within a batch of 10,000 devices.

Also, considering the fact that people have noticed that the X-Y coordinates are rather inconsistent and may contain a lot of 0's in the MSB, especially for chips with large wafer size, which means lot number + coordinate data is not unique at all.

CityHash Code for Generating 32-bit STM32 Unique ID

A hashing function is the perfect solution to this problem. A hashing function maps the input data string to a (usually) shorter data string while trying to distribute the mapping as evenly as possible.
With a good hashing function, a single bit changing in the input string will give you a completely different output string. The main characteristic that we are looking for in the hashing function here is that the **repitition in the output string should be evenly distributed and minimized, given a changing longer input string**.

Google's CityHash is a great solution to the above problem. The string hashing logic is simple, computationally light weight, low footprint and easy to use. CityHash is optimized for hashing strings.

The following C code (H file and C file) is enough to get the job done. You simply need to feed in a string with 12 bytes and the CityHash algorithm produces a unique 32-bit UID from the STM32 UID register contents.

Note that this is not the complete implementation of the CityHash logic. I have removed several conditional compilation clauses and the logic that works on longer data input strings in order to save flash space.

This code will only work on 5-byte to 12-byte long input fields.

Header file (**stm32_uidhash.h**):

```h
#ifndef _STM32_UIDHASH_
#define _STM32_UIDHASH_

uint32_t Hash32Len5to12(const char *s, size_t len);

#endif
```

C file (**stm32_uidhash.c**):

```c
#include <stdio.h>
#include <string.h>

#include "stm32_uidhash.h"

// Magic numbers for 32-bit hashing.  Copied from Murmur3.
static const uint32_t c1 = 0xcc9e2d51;
static const uint32_t c2 = 0x1b873593;

static uint32_t UNALIGNED_LOAD32(const char *p) {
  uint32_t result;
  memcpy(&result, p, sizeof(result));
  return result;
}

static uint32_t Fetch32(const char *p) {
  return UNALIGNED_LOAD32(p);
}

static uint32_t Rotate32(uint32_t val, int shift) {
  // Avoid shifting by 32: doing so yields an undefined result.
  return shift == 0 ? val : ((val >> shift) | (val << (32 - shift)));
}

// A 32-bit to 32-bit integer hash copied from Murmur3.
static uint32_t fmix(uint32_t h)
{
```

```
  h ^= h >> 16;
  h *= 0x85ebca6b;
  h ^= h >> 13;
  h *= 0xc2b2ae35;
  h ^= h >> 16;
  return h;
}

static uint32_t Mur(uint32_t a, uint32_t h) {
  // Helper from Murmur3 for combining two 32-bit values.
  a *= c1;
  a = Rotate32(a, 17);
  a *= c2;
  h ^= a;
  h = Rotate32(h, 19);
  return h * 5 + 0xe6546b64;
}

uint32_t Hash32Len5to12(const char *s, size_t len) {
  uint32_t a = (uint32_t)len, b = a * 5, c = 9, d = b;
  a += Fetch32(s);
  b += Fetch32(s + len - 4);
  c += Fetch32(s + ((len >> 1) & 4));
  return fmix(Mur(c, Mur(b, Mur(a, d))));
}
```

Using the code is easy. All you need to do is pass an array with STM32 unique ID register values as a 12-character array.

The value returned is the 32-bit hashed UID.

Here is a code snippet showing how to use the CityHash function on an **STM32G4**:

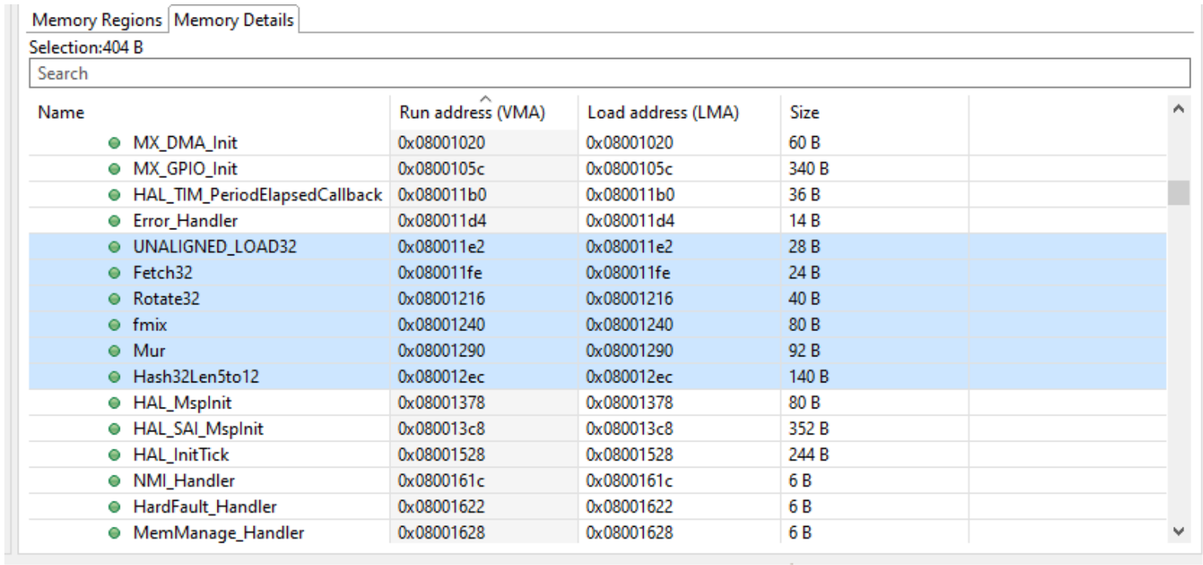
```
char uidstr[12];

// Arrange 12 bytes of UID into uidstr[]
uint32_t uid = LL_GetUID_Word0 ();
memcpy (&uidstr[8], &uid, 4);
```

```
6   uid = LL_GetUID_Word1 ();
7   memcpy (&uidstr[4], &uid, 4);
8   uid = LL_GetUID_Word2 ();
9   memcpy (&uidstr[0], &uid, 4);
10
11  // Generate UID value from uidstr[]
12  uid = Hash32Len5to12 ((const char *)uidstr, 12);
```

The code footprint is tiny, only about 400 bytes even with optimization level set to debug.

For comparison, HAL peripheral init routines are usually larger than the hashing logic.



Generating 32-bit STM32 Unique ID, CityHash Flash Usage

Tried and Tested on our Products

We have a bunch of products out there that use this hashing algorithm to reduce the 96 bits down to a short 32-bit STM32 unique ID. The PCB Artists sound level meter is one such example. The device contains a 32-bit unique ID that it generates from the 96-bit factory programmed UID.

As we determined above, there is **little to no chance of having the same 32-bit STM32 unique ID in a lot of around 10,000 devices**, especially when they are parts from the same manufacturer reel with consecutive serial numbers.

Have Something to Say?

Feel free to ask away via the **Live Chat**, drop us a message using the **Quick Contact** form in the sidebar, or leave a **comment** below.

Change Log

- **24 May 2023**
  – Initial release

STM32

1 comment 1

YOU MAY ALSO LIKE

SOUND LEVEL SENSOR MODULE FOR ARDUINO

USING 4 LANE MIPI DSI DISPLAY WITH STM32

SOLVED: UNDEFINED REFERENCE TO FUNCTION (STM32 CUBEIDE)

LEAVE A COMMENT

Your Comment

Name*

Email*

Website

Save my name, email, and website in this browser for the next time I comment.

Please enter an answer in digits:

four + three =

SUBMIT

1 COMMENT

EERO AF HEURLIN REPLY

January 17, 2025 - 12:07 pm

Many thanks for this code, I made a port to MicroPython it's published here https://gist.github.com/rambo/018b55fb82dad66ac8a8188b2b7c59d1